



Università
di Catania

DIPARTIMENTO DI MATEMATICA E INFORMATICA
CORSO DI LAUREA MAGISTRALE IN INFORMATICA

Edoardo Tantari, Raffaele Terracino

Multi-Object Tracking di veicoli con ByteTrack e PuTR:
benchmark e confronto

RELAZIONE PROGETTO DI
COMPUTER VISION

Prof: Sebastiano Battiato
Prof: Francesco Guarnera

Anno Accademico 2025 - 2026

Indice

1	Introduzione	3
1.1	Il problema del tracking	3
1.2	Scopo del progetto	4
2	Dataset	6
2.1	Il dataset UA-DETRAC	6
2.1.1	Ignored regions	7
2.1.2	Accorpamento delle classi	8
2.2	Pre-processing	8
2.2.1	Conversione per YOLO	8
2.2.2	Ground truth per TrackEval	9
2.2.3	Conversione per PuTR	10
2.2.4	Riepilogo	11
3	Metodologia	13
3.1	Object Detection	13
3.2	ByteTrack	15
3.3	PuTR	16
3.4	Metriche del MOT	17
3.5	Configurazione utilizzata	19

<i>INDICE</i>	2
4 Risultati	20
4.1 Model selection	20
4.1.1 Detector	20
4.1.2 ByteTrack	21
4.1.3 PuTR	21
4.2 Benchmark di confronto	22
4.3 Risultati qualitativi	23
5 Conclusione	25
Bibliografia	26

Capitolo 1

Introduzione

1.1 Il problema del tracking

Il problema del visual tracking consiste nel mantenere l'identità di uno o più oggetti lungo una sequenza video. Mentre un detector lavora sul singolo frame e restituisce un insieme di bounding box non correlati tra loro, un tracker deve stabilire una corrispondenza temporale: il box che nel frame t racchiude un certo oggetto deve essere associata al box che racchiude lo stesso oggetto nel frame $t + 1$, e così via fino a quando l'oggetto esce dalla scena. L'output non è più un insieme di rilevazioni, ma un insieme di traiettorie, ciascuna identificata da un ID persistente.

La distinzione classica è tra *Single Object Tracking* (SOT) e *Multi Object Tracking* (MOT). Nel primo caso il target è unico e noto, venendo inizializzato nel primo frame e seguito idealmente fino alla sua uscita dall'inquadratura. Nel MOT, invece, i target sono multipli e il loro numero, non noto a priori, varia nel tempo; gli oggetti entrano ed escono dal campo visivo, si occludono a vicenda, possono essere temporaneamente nascosti da elementi statici della scena e riapparire in una posizione diversa da quella prevista.

Il paradigma dominante nella letteratura recente è il tracking by detection, dove ad ogni frame viene applicato un detector, e le rilevazioni prodotte vengono associate alle tracce già esistenti tramite un modulo di associazione. La qualità finale dipende quindi da due fattori distinti: la bontà del detector e la bontà dell'associazione. Uno dei problemi metodologici tipici nel confronto fra tracker è proprio separare i contributi dei due componenti.

Gli errori tipici nel campo del MOT sono gli ID switch, ovvero scambio di identità tra due tracce, oltre che la frammentazione, cioè l'interruzione di una traiettoria che viene poi ripresa con un nuovo identificativo.

Applicazioni tipiche del tracking sui veicoli sono la gestione adattiva dei semafori sulla base della domanda reale, la rilevazione automatica di incidenti e code, l'analisi statistica dei flussi per la pianificazione infrastrutturale e il supporto alla guida autonoma e ai sistemi di assistenza alla guida.

1.2 Scopo del progetto

L'obiettivo di questo lavoro è confrontare, a parità di detector, due famiglie di approcci al modulo di associazione:

- **Detector e algoritmo di associazione**, dove le identità vengono propagate da una procedura esplicita e non addestrata, basata su predizione del moto e matching tra predizioni e rilevazioni.
- **Detector e transformer**, dove l'associazione è demandata a un modello appreso, che sfrutta meccanismi di attenzione per mettere in relazione le rilevazioni correnti con lo stato delle tracce.

Come rappresentante della prima famiglia è stato scelto **ByteTrack** [1]. Lo stato di ogni traccia viene propagato con un filtro di Kalman a moto lineare

e l'associazione avviene in due passaggi: prima le rilevazioni ad alta confidenza vengono abbinate alle predizioni tramite costo basato su IoU (Intersection over union) e algoritmo ungherese, poi quelle a bassa confidenza vengono confrontate con le sole tracce rimaste libere. L'idea alla base è l'uso di due soglie invece di una: le rilevazioni sopra la soglia alta sono considerate affidabili e associate per prime, mentre quelle nella fascia intermedia, solitamente scartate, sono usate in un secondo giro di associazione perché spesso corrispondono a veicoli occlusi o sfocati, e recuperarle evita di spezzare la traccia. Non vengono usati descrittori di apparenza, ovvero vettori di feature che descrivono l'aspetto visivo dell'oggetto contenuto in un bounding box, quindi l'associazione si basa solo su informazione geometrica.

Come rappresentante della seconda famiglia è stato scelto **PuTR** [2]. Anche qui detection e associazione restano due moduli separati, ma la regola di matching viene sostituita da un transformer che lavora sulle coordinate delle box e non sui pixel. Il funzionamento è un problema di sequenze: le box rilevate negli ultimi frame vengono messe in fila come una lista di token, similmente alle parole di una frase, e il transformer le legge tutte insieme con un meccanismo di attenzione che permette a ogni box di guardare solo quelle dei frame precedenti. Da questa lettura il modello produce per ogni box un vettore di feature che tiene conto del contesto, cioè di come si sono mosse le altre box nella finestra temporale considerata. Confrontando questi vettori delle rilevazioni correnti con quelle delle tracce attive si ottiene un punteggio di somiglianza per ogni coppia, e l'assegnamento finale viene deciso con l'algoritmo ungherese come in ByteTrack. La differenza sostanziale è che il criterio di somiglianza non è più una formula fissa come l'IoU ma qualcosa che il modello ha imparato dai dati. Anche PuTR non usa l'apparenza, quindi i due tracker differiscono solamente per come elaborano la stessa informazione.

Capitolo 2

Dataset

2.1 Il dataset UA-DETRAC

UA-DETRAC [3] è un benchmark per la detection e il tracking di veicoli realizzato da un consorzio di ricerca tra varie università. I video sono stati acquisiti con una Canon EOS 550D in 24 diverse località di Pechino e Tianjin, in prevalenza da cavalcavia e passerelle sopra la carreggiata, quindi con camera fissa e inquadratura dall'alto. Il benchmark comprende 100 sequenze estratte da circa 10 ore di ripresa, per oltre 140.000 frame e circa 1,21 milioni di bounding box annotate, riferite a 8.250 veicoli distinti. I video sono registrati a 25 fps e ogni frame è memorizzato come immagine JPEG a risoluzione 960×540 .

Le sequenze coprono situazioni di traffico eterogenee, dall'autostrada urbana agli incroci semaforizzati, e quattro condizioni di illuminazione e meteo (soleggiato, nuvoloso, pioggia, notte). Ogni veicolo è annotato con un identificativo persistente lungo tutta la sequenza, con la categoria (car, bus, van, others) e con attributi aggiuntivi come il livello di occlusione e di troncamento. La suddivisione ufficiale prevede 60 sequenze di training e 40 di test, riprese in

località diverse tra loro in modo da ridurre il rischio che un modello si specializzi su una particolare scena.

Il formato di distribuzione è a livello di sequenza: le immagini sono organizzate in una cartella per video, con nomi progressivi del tipo `img00001.jpg`, mentre le annotazioni sono un unico file XML per video. All'interno dell'XML ogni elemento `frame` contiene la lista dei `target` presenti in quel frame, ciascuno con il proprio `id` e una `box` espressa come `left`, `top`, `width`, `height` in coordinate assolute. Lo stesso file contiene anche l'elenco delle *ignored regions* della sequenza.

2.1.1 Ignored regions

Le ignored regions sono rettangoli che delimitano porzioni di scena che gli autori hanno deciso di non annotare. Si tratta tipicamente di corsie molto lontane dalla camera, dove i veicoli occupano pochi pixel e non sarebbero etichettabili in modo affidabile, oppure di aree laterali come parcheggi o strade di servizio che non fanno parte del flusso di interesse. In queste regioni i veicoli ci sono, ma nell'insieme di ground truth non compaiono.

Questo crea un problema in fase di valutazione. Un detector addestrato correttamente rileva anche i veicoli nelle zone ignorate, ma quelle rilevazioni non trovano corrispondenza nel ground truth e vengono conteggiate come falsi positivi, con un effetto a cascata sulle metriche di tracking. Per questo motivo le ignored regions vengono estratte dagli XML e salvate in un file `ignored_regions.json`, che viene poi usato in fase di inferenza per scartare le predizioni che ricadono al loro interno prima di passarle al tracker. Il ground truth resta invariato, il filtro agisce solo sull'output del detector.

2.1.2 Accorpamento delle classi

Le quattro categorie originali sono state compattate in un'unica classe `vehicle`. La motivazione principale per cui è stata effettuata questa scelta è che nessuno dei due tracker considerati usa l'etichetta di classe come informazione per l'associazione, e l'obiettivo del progetto riguarda la qualità della localizzazione e dell'associazione, non il riconoscimento della tipologia di veicolo.

2.2 Pre-processing

Il dataset originale non è direttamente utilizzabile da nessuno dei componenti della pipeline, per cui sono stati scritti tre script di conversione, uno per il detector *YOLO*, uno per il tracker *PuTR* e uno per *TrackEval* [4], tool di evaluation descritto successivamente. Tutti condividono la stessa logica di suddivisione delle sequenze e lo stesso seed, in modo che le stesse sequenze finiscano sempre nello stesso split.

2.2.1 Conversione per YOLO

Lo script `detrac_to_yolo.py` produce il dataset nel formato richiesto da Ultralytics, cioè una cartella di immagini e una di label parallele, con un file di testo per immagine. Ogni riga di label contiene l'indice di classe e le coordinate della box normalizzate rispetto alle dimensioni dell'immagine, nella forma (x_c, y_c, w, h) . La conversione dalle coordinate DETRAC prevede il passaggio da angolo superiore sinistro e dimensioni a centro e dimensioni, la normalizzazione per 960 e 540 e il clipping ai bordi dell'immagine, necessario perché alcune annotazioni sporgono oltre il frame per veicoli parzialmente usciti dall'inquadratura. I box che dopo il clipping risultano degeneri vengono scartati.

La suddivisione fra training e validation avviene a livello di sequenza e non di frame. Frame consecutivi dello stesso video sono quasi identici, quindi uno split casuale sui singoli frame metterebbe immagini quasi uguali da entrambi i lati, producendo una stima di validazione ottimistica e sostanzialmente priva di significato. Le 60 sequenze di training ufficiali sono quindi mescolate con seed fisso e divise in 80% training e 20% validation, ottenendo 48 sequenze per l'addestramento e 12 per la validazione. Le 40 sequenze di test ufficiali restano intatte e sono usate solo per la valutazione finale.

Sul numero di frame interviene invece il sottocampionamento. A 25 fps due frame consecutivi contengono quasi la stessa informazione, per cui usarli tutti significa moltiplicare il tempo di addestramento senza aumentare la varietà dei dati, con il rischio concreto di overfitting su singole scene. È stato verificato empiricamente che le sessioni di training del detector, sulle intere sequenze del dataset, portassero a tempi di esecuzione infattibili per epoca, motivo per cui è stato adottato un sample rate fisso pari a 10, cioè viene tenuto un frame ogni dieci, il che corrisponde a un frame ogni 0,4 secondi. Il valore è fissato una volta per tutte e usato in modo identico in tutti gli esperimenti di detection, così che il confronto tra configurazioni di training non sia influenzato dalla quantità di dati. Il campionamento viene effettuato su training e validation set.

2.2.2 Ground truth per TrackEval

La valutazione del tracking è affidata a TrackEval [4], che si aspetta i dati nel formato MOTChallenge. Lo script `detrack_to_mot.py` genera per ogni sequenza un file `gt/gt.txt` con una riga per ogni box annotata, nel formato a nove campi

```
frame, id, left, top, width, height, conf, class,
visibility
```

dove gli ultimi tre campi sono posti a 1 perché il pre-processing di TrackEval, pensato per gestire le classi e le occlusioni di MOT17, non serve in questo contesto. Accanto al ground truth viene scritto un file `seqinfo.ini` con nome della sequenza, frame rate, lunghezza e dimensioni delle immagini, e per ogni split un *seqmap* che elenca le sequenze da valutare.

Qui il ground truth non viene sottocampionato. La ragione è che il tracking va valutato nelle condizioni operative reali, cioè a 25 fps: eliminare nove frame su dieci renderebbe gli spostamenti tra frame consecutivi molto più ampi, complicando artificialmente l'associazione e falsando il confronto. Ne segue la distinzione tra i due usi del validation set. Per la detection il validation set è sottocampionato come il training, perché serve soltanto a confrontare configurazioni del detector e la metrica di riferimento è calcolata su frame indipendenti. Per il tracking lo stesso validation set viene invece usato per intero, sequenza per sequenza, perché la continuità temporale è proprio l'oggetto della misura. Il test set è sempre usato per intero.

2.2.3 Conversione per PuTR

Lo script `detrac_to_putr.py` prepara i dati nel formato che PuTR si aspetta, che è a sua volta derivato da MOT17 ma con vincoli più stringenti sulla numerazione. Il loader del modello richiede infatti che i frame di ogni sequenza siano numerati in modo contiguo a partire da 1 e con nome a sei cifre, mentre in DETRAC la numerazione parte dai nomi originali e, dopo il sottocampionamento, presenta salti. Lo script costruisce quindi una mappa dai numeri di frame originali a quelli nuovi, rinomina le immagini di conseguenza e riscrive il ground truth applicando la stessa mappa, in modo che annotazioni

e immagini restino allineate.

Anche qui il sottocampionamento è applicato al solo split di training, con lo stesso sample rate di 10, mentre validation e test sono usati per intero. Di conseguenza il frame rate dichiarato nel `seqinfo.ini` delle sequenze di training è $25/10 = 2,5$ fps: è un dettaglio non trascurabile, perché PuTR deve imparare ad associare box che si spostano di quantità realistiche per l'intervallo temporale considerato.

Una differenza rispetto agli altri due script è la presenza della cartella `det`, che contiene le rilevazioni prodotte dal detector già addestrato. PuTR infatti non esegue detection, si limita all'associazione, quindi in validazione e in test riceve in ingresso le box del nostro YOLO invece di calcolarle. Questo è anche il meccanismo che garantisce l'equità del confronto discusso nel capitolo precedente, ovvero che le rilevazioni date in pasto a PuTR sono esattamente le stesse che riceve ByteTrack.

2.2.4 Riepilogo

La Tabella 2.1 riassume la composizione finale dei tre split e il numero di immagini effettivamente utilizzate.

Split	Sequenze	Frame totali	Frame usati (detection)	Frame usati (tracking)
Train	48	68150	6815	6815
Val	12	14170	1417	14170
Test	40	56167	—	56167

Tabella 2.1: Composizione degli split dopo il pre-processing. Il sottocampionamento con sample rate 10 è applicato al training per entrambi i task e al validation per la sola detection.

La Figura 2.1 mostra alcuni frame rappresentativi del dataset, scelti in modo da coprire condizioni di illuminazione e meteo differenti.

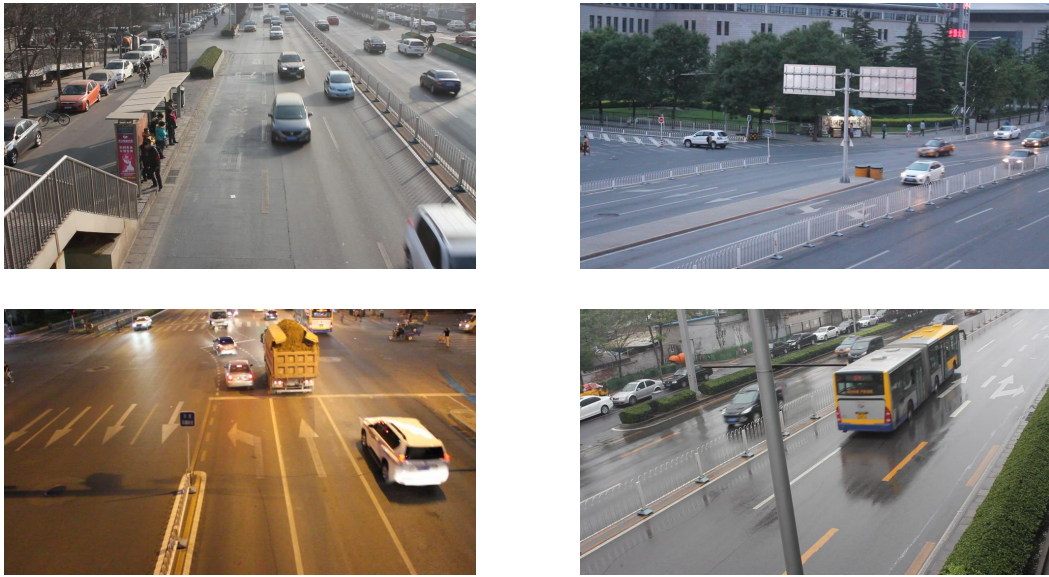


Figura 2.1: Esempi di frame di sequenze in differenti condizioni di illuminazione e meteo

Capitolo 3

Metodologia

3.1 Object Detection

Il modello di Object Detection scelto per il progetto è l'ultima versione di YOLO, detta YOLO26 [5], che viene utilizzata nella sua versione "medium" pre-trained sul dataset COCO [6]. L'obiettivo di questa fase è costruire un buon detector, effettuando un *fine-tuning* di YOLO26 sul dataset DETRAC mediante la libreria Ultralytics ¹, con lo scopo di fornire a ByteTrack [1] e Pu-TR [2] le stesse detection per la fase di validazione e confronto finale sul test set. La base di partenza è stata costruire una baseline, prendendo il modello e allenandolo sul dataset secondo la configurazione di default di iperparametri e data augmentation. Vengono effettuati un totale di 3 esperimenti, ognuno corrispondente a una diversa configurazione di iperparametri del modello, scegliendo il miglior modello seguendo la prassi della *model selection*. Per ogni esperimento, si valuta il modello addestrato calcolandone le principali metriche sul validation set, scegliendo il modello con le metriche più alte. Il test set non viene utilizzato in questa fase. La tabella 3.1 contiene la configurazione

¹<https://docs.ultralytics.com>

di iperparametri per ogni esperimento. Il primo esperimento rappresenta la *baseline* per l’object detection su DETRAC e consiste nell’addestramento di YOLO26 secondo la configurazione di default di iperparametri e data augmentation. A partire da questa, si è ragionato sulla natura del dataset: poichè vi è solo una classe, la loss di classificazione *cls* può essere pesata di meno, a favore di un aumento della *box loss*, ben più importante per il tracking. Nel terzo esperimento si è abbassata ulteriormente la *cls loss*, aumentando invece la *dfl loss*, mantenendo invariata la *box loss*. Non vengono riportati gli iperparametri che sono rimasti invariati rispetto alla configurazione di default della libreria. Si effettua model selection calcolando le metriche tipiche del-

Iperparametro	Esperimento 1	Esperimento 2	Esperimento 3
Training			
epochs	50	100	100
batch	16	12	12
patience	5	30	30
cache	false	disk	disk
Loss			
box	7.5	10.0	10.0
cls	0.5	0.5	0.1
dfl	1.5	1.5	3.5
Data augmentation			
close_mosaic	10	20	20
scale	0.5	0.4	0.4
hsv_s	0.7	0.5	0.5
hsv_v	0.4	0.5	0.5

Tabella 3.1: Configurazione degli iperparametri per ogni esperimento

l’object detection e scegliendo il modello con le migliori performance, usando come metrica principale la *MAP@50–95*. Con il miglior modello si effettua inferenza sul validation set non campionato e sul test set, filtrando le *ignored regions* di DETRAC e utilizzando una *confidence* pari a 0.05. Le detection

vengono quindi salvate su disco, pronte per essere utilizzate da ByteTrack e PuTR. Il codice per il training e la model selection è implementato come notebook *yolo_training.ipynb*, mentre il salvataggio (dump) delle detection si effettua mediante lo script *dump_detections.py*.

3.2 ByteTrack

Sono stati effettuati 3 esperimenti per ByteTrack, ognuno relativo a una diversa configurazione degli iperparametri specifici dell'algoritmo, per effettuare model selection sul validation set. La tabella 3.2 mostra le configurazioni di iperparametri scelte, fornendo per ognuno una descrizione del suo significato. L'esperimento 1 consiste nella configurazione di default dei parametri di ByteTrack, similmente a quanto effettuato con YOLO. A partire dai risultati di questo modello sul validation set, si è deciso di incrementare *new_track_tresh* a 0.5, considerando 0.25 troppo restrittivo. Analizzando le metriche, nel terzo esperimento, si è tornati alla configurazione di default decidendo di abbassare *track_buffer* a 15. La configurazione che ha portato alle migliori metriche viene scelta per la fase finale di confronto.

Parametro	Descrizione	Exp. 1	Exp. 2	Exp. 3
<i>track_high_thresh</i>	Soglia per la prima associazione	0.25	0.25	0.25
<i>track_low_thresh</i>	Soglia per la seconda associazione	0.1	0.1	0.1
<i>new_track_thresh</i> [†]	Soglia per creare una nuova traccia	0.25	0.5	0.25
<i>track_buffer</i> [†]	Frame di vita di una traccia persa	30	30	15
<i>match_thresh</i>	Soglia di similarità (IoU) nel matching	0.8	0.8	0.8
<i>fuse_score</i>	Fusione di confidenza e IoU nel costo	true	true	true

Tabella 3.2: Configurazioni di iperparametri di ByteTrack utilizzate nei tre esperimenti

3.3 PuTR

Il modello PuTR è stato addestrato sul training set campionato di DETRAC, modificando il codice dove necessario per renderlo compatibile col dataset. Il codice aggiornato e il file di configurazione per il training è descritto nel repository del progetto ². PuTr utilizza due insiemi di iperparametri: uno per il training, fissato per tutto il progetto, ed uno per l'inferenza, che cambia nella fase di model selection. Viene addestrato PuTr per 6 epoche, monitorando l'andamento della loss durante il training. Viene fatta una model selection preliminare sui checkpoint della quarta, quinta e sesta epoca, con la configurazione di iperparametri di inferenza di default. Scelto il miglior checkpoint, si effettua la model selection vera e propria effettuando, sulla linea di bytetrack, altri due esperimenti aggiuntivi rispetto alla configurazione di default di inferenza (Esperimento 1). La tabella 3.3 riporta la configurazione di iperparametri di inferenza per gli esperimenti, oltre che una breve descrizione per ognuno, evidenziando in grassetto le differenze. Gli esperimenti sono condotti sulla riga di ByteTrack: nel secondo esperimento si abbassa *TRACK_SCORE_TRESH*, considerando 0.7 troppo alta, mentre nel terzo, mantenendo fissa tale soglia, si alza invece la prima soglia di associazione *ASSO_THRE1*. Il miglior modello sul validation set costituisce l'output di questa fase, che rende PuTr pronto per il benchmark finale contro ByteTrack.

²<https://github.com/eddy2809/CV-Tracking>

Parametro	Descrizione	Exp. 1	Exp. 2	Exp. 3
DET_SCORE.THRESH	Soglia di confidenza per filtrare le detection	0.1	0.1	0.1
TRACK_SCORE.THRESH [†]	Soglia per inizializzare una nuova traccia	0.7	0.5	0.5
MISS.TOLERANCE	Frame di vita di una traccia persa	30	30	30
MAX.NFRAMES	Lunghezza della sequenza in input al Transformer	30	30	30
MIN.TRACK.HITS [‡]	Detection minime per confermare una traccia	3	3	3
NEW.TRK.TOLERANCE [‡]	Tolleranza per le tracce in stato <i>New</i>	3	3	3
DETS.IOU.THRESH [‡]	Soglia IoU per la soppressione tra detection	0.4	0.4	0.4
ASSO.THRE1 [†]	Prima soglia di associazione	0.2	0.2	0.4
ASSO.THRE2	Seconda soglia di associazione	0.8	0.8	0.8

Tabella 3.3: Configurazioni di inferenza di PuTR utilizzate nei tre esperimenti sul dataset DETRAC. I parametri in grassetto sono quelli che cambiano fra gli esperimenti.

3.4 Metriche del MOT

Per la valutazione dei modelli di MOT si utilizza la famosa libreria TrackEval [4], standard per il calcolo delle metriche per il MOT. La libreria implementa le più importanti metriche utilizzate nella valutazione del task. La prima di queste è HOTA (Higher Order Tracking Accuracy), che costituisce la metrica principe per la valutazione. La difficoltà inerente del problema del tracking è che ogni box predetto ha anche un ID, che deve essere coerente nel tempo per lo stesso oggetto, frame dopo frame. Dopo l'inferenza, ogni frame ha N predizioni da assegnare a M ground truth. Per effettuare l'assegnamento, la libreria utilizza l'algoritmo ungherese ottimizzando, fissata una soglia τ , l'Intersection over Union (IoU) totale. Il risultato dell'assegnamento consiste in tre insiemi di predizioni: True Positive (TP_τ), predizioni associate alla ground truth tali che $IoU > \tau$, False Positive (FP_τ), predizioni senza corrispondenza nella ground truth e, infine, False Negative (FN_τ), ovvero ground truth che il modello non ha individuato. A partire dai risultati dell'assegnamento, si calcolano due metriche: la Detection Accuracy $DetA_\tau$, che indica quanto bene il modello rileva gli oggetti, e la Association Accuracy $AssA_\tau$, che indica quanto l'associazione è coerente nel tempo (ovvero, quanto bene si mantiene

lo stesso ID sullo stesso oggetto nel tempo). Calcolate le due metriche, si può calcolare $HOTA_\tau$ come:

$$HOTA_\tau = \sqrt{DetA_\tau * AssA_\tau} \quad (3.1)$$

L'HOTA finale si ottiene calcolando e mediando $AssA_\tau$, $DetA_\tau$ e $HOTA_\tau$ per ogni soglia τ ed è un indicatore di quanto il tracking funziona bene in generale, raccogliendo in un unico valore i concetti intuitivi di "quanto si rilevano bene gli oggetti" e di "quanto si mantengono nel tempo". HOTA è una metrica da massimizzare e ha valore massimo pari a 1. Le altre metriche considerate nella valutazione del tracking sono le seguenti:

- IDSW: frequenza totale per cui è accaduto che un oggetto avente un certo ID al frame t ha avuto un cambio di ID al frame $t + 1$
- MOTA (Multiple Object Tracking Accuracy): metrica storica per la valutazione del MOT, è un conteggio complessivo degli errori rispetto al numero di ground truth;
- MOTP (Multiple Object Tracking Precision): indicatore della qualità di localizzazione dei veri positivi;
- IDF1 (Identification F_1 score): indica quanto, in media, il tracker mantiene lo stesso ID su un oggetto per tutta la sua durata;
- Frag (Fragmentation): numero di volte in cui una traiettoria viene interrotta. Rappresenta una misura della continuità del tracking;
- MT (Mostly Tracked): percentuale di traiettorie ground truth tracciate correttamente per almeno l'80% della loro durata

- ML (Mostly Losted): opposto di MT, indica la percentuale di traiettorie ground truth tracciate correttamente per meno del 20% della loro durata.

Per HOTA, DetA, AssA, MOTA, MOTP, IDF1 e MT è preferibile un valore più alto, mentre per IDSW, Frag e ML un valore più basso.

3.5 Configurazione utilizzata

La seguente lista mostra la configurazione utilizzata per il training e valutazione dei modelli descritti.

Processore (CPU): AMD Ryzen 7 9800X3D (5.2 GHz, 8 Core)

Memoria RAM: 64 GB DDR5

Scheda Video (GPU): NVIDIA RTX 4090 (24 GB VRAM)

Sistema Operativo: Windows 11

Capitolo 4

Risultati

4.1 Model selection

4.1.1 Detector

La tabella 4.1 mostra le metriche calcolate sul validation set campionato con $sample_rate = 10$, per ognuno degli esperimenti descritti. La baseline, ovvero l'esperimento 1, risulta già essere un buon detector, con una buona $mAP@50$ e una solida $mAP@50-95$. Aumentare la box_loss e variare leggermente la data augmentation ha avuto l'effetto desiderato, facendo aumentare le metriche di 1-2 punti. Il terzo esperimento, effettuato sulla linea del secondo, fa schizzare la precision, che raggiunge il 90% e fa salire di 2 punti la $mAP@50-95$ rispetto all'esperimento 2. Dalle metriche risulta chiaro che il terzo esperimento vince nettamente sugli altri due, confermando l'ipotesi che aumentare i pesi della box_loss e della dfl_loss avrebbe potuto beneficiare alle performance. Viene pertanto scelto il modello relativo al terzo esperimento, con cui viene effettuato il dump delle detection sul validation set completo e sul test set.

Configurazione	Precision	Recall	mAP@50	mAP@50–95
Exp. 1	0.8545	0.8249	0.8442	0.6183
Exp. 2	0.8729	0.8298	0.8605	0.6271
Exp. 3	0.9079	0.8375	0.8709	0.6457

Tabella 4.1: Metriche dell’Object Detection con YOLO, calcolate sul validation set campionato. In grassetto il valore migliore per ciascuna metrica.

4.1.2 ByteTrack

La tabella 4.2 riporta i risultati della fase di model selection sul validation set per ogni esperimento effettuato. La variazione di HOTA è minima fra le tre configurazioni. Nel secondo esperimento, aumentare *new_track_tresh* ha avuto l’effetto desiderato di abbassare del 19% gli ID switch (IDSW) al costo di una differenza di 0.35 punti HOTA rispetto all’esperimento 1. Guardando soltanto le metriche principali, ovvero HOTA, DetA ed AssA, l’esperimento 3 risulta essere, marginalmente, quello con le migliori performance. Viene pertanto scelto questo modello per la parte di benchmark finale.

Configurazoin	HOTA	DetA	AssA	MOTA	MOTP	IDF1	IDSW	Frag	MT	ML
Exp.1	64.465	61.030	68.531	65.992	82.470	81.626	85	467	771	50
Exp.2	64.124	60.750	68.120	65.887	82.456	81.508	69	448	752	50
Exp.3	64.473	61.033	68.547	65.993	82.475	81.623	86	461	770	51

Tabella 4.2: Metriche del MOT calcolate sul validation set per le tre configurazioni di ByteTrack. In grassetto il valore migliore per ciascuna metrica.

4.1.3 PuTR

La tabella 4.3 riporta i risultati degli esperimenti condotti per PuTR, suddividendoli in due categorie: la model selection preliminare sui checkpoint e quella effettiva sul checkpoint scelto. I tre checkpoint presentano variazioni minime, pertanto la scelta risulta essere arbitraria. Viene scelto il quinto checkpoint, a

causa dei minimi miglioramenti sulle metriche. Su questo, vengono effettuati i due esperimenti descritti in 3.3. L'esperimento 2 migliora marginalmente le prestazioni della baseline (il checkpoint 5) incrementando gli identity switch a 50. L'esperimento 3 consegue ulteriori, marginali, miglioramenti HOTA e DetA, presentando però un numero elevatissimo di identity switch, pari a 92. La fase di model selection si conclude quindi con la selezione del quinto checkpoint (*chkpt_5*) con la configurazione di default di iperparametri di inferenza.

Esperimento	HOTA	DetA	AssA	MOTA	MOTP	IDF1	IDSW	Frag	MT	ML
<i>Selezione del checkpoint (Exp. 1)</i>										
ckpt_4	71.963	69.276	75.019	78.995	86.914	88.476	34	253	1047	27
ckpt_5	71.969	69.280	75.025	79.006	86.912	88.479	37	250	1048	27
ckpt_6	71.965	69.278	75.021	78.998	86.913	88.478	34	253	1047	27
<i>Configurazioni di inferenza su ckpt_5</i>										
Exp. 2	71.976	69.321	75.002	78.671	86.824	88.192	50	295	1078	22
Exp. 3	72.026	69.636	74.775	79.167	86.858	87.658	92	277	1072	26

Tabella 4.3: Risultati di tracking di PuTR sul dataset DETRAC. Il blocco superiore confronta tre checkpoint del modello valutati con la configurazione di inferenza di default; il blocco inferiore riporta i due esperimenti condotti sul checkpoint migliore. In grassetto il valore migliore per ciascuna metrica.

4.2 Benchmark di confronto

Effettuata la fase di model selection per Bytetrack e PuTR, si caricano i migliori checkpoint e si fa inferenza sul test set, ottenendo i risultati della tabella 4.4. Dal benchmark si nota che il vincitore è PuTR, che sorpassa di 3 punti in HOTA ByteTrack. La potenza del metodo si vede anche dagli identity switch: ByteTrack ne fa 1005, PuTR invece ne fa 339, ovvero il 66,3% in meno. L'accuracy di associazione (AssA) incrementa di 5 punti rispetto a PuTR, segnale che il modello restituisce associazioni molto più coerenti nel tempo, coerentemente con il numero di IDSW. L'unica metrica in cui ByteTrack è migliore è la ML, risultato che non influisce sulla valutazione generale

dei modelli. L’approccio con un trasformer di PuTR vince nettamente su ByteTrack, concludendo così il benchmark sul dataset DETRAC.

Metodo	HOTA	DetA	AssA	MOTA	MOTP	IDF1	IDSW	Frag	MT	ML
ByteTrack	67.213	63.308	71.684	70.882	86.354	78.540	1005	2702	1494	131
PuTR	70.795	65.462	76.838	73.684	87.942	82.644	339	2283	1570	166

Tabella 4.4: Confronto sul test set tra la migliore configurazione di ByteTrack e la migliore configurazione di PuTR. In grassetto il valore migliore per ciascuna metrica.

4.3 Risultati qualitativi

La figura 4.1 riporta alcuni esempi di inferenza di PuTR su una clip di 4 frame, estratta dal test set guardando i risultati delle metriche su ogni sequenza. Le immagini sono realizzate con lo script *draw.py* a partire dai risultati dell’inferenza di PuTr sul test set e mostrano, per ogni frame, il numero di ground truth (GT), predizioni (pred), True Positive (TP) e False Negative (FN). Dalle figure è evidente che il principale limite del modello è non riuscire a rilevare e tracciare veicoli se sono molto vicini tra loro, come mostra la porzione in alto a sinistra del primo frame. Si nota anche che alcuni veicoli non vengono rilevati al primo frame, ma nei successivi tre sono rilevati e tracciati correttamente, risultato coerente con la *DetA* calcolata. Considerando il limitato numero di frame per il training, si può dire che il modello pertanto funziona bene ma ha difficoltà nel rilevare oggetti piccoli e occlusi.



Figura 4.1: Esempi di risultati qualitativi prodotti dal miglior modello PuTr, vincitore del benchmark, su una porzione di clip di test consistente in 4 frame. In giallo sono evidenziati i miss, in verde i box ground truth e in rosso le predizioni.

Capitolo 5

Conclusione

Il progetto ha dimostrato la superiorità del modello PuTR [2] rispetto all'algoritmo ByteTrack [1] a parità di detection. I risultati qualitativi mostrano che entrambi i modelli sembrano essere limitati dalla qualità delle detection, fatto dimostrato anche dai risultati della metrica DetA sul test set. Sviluppi futuri del progetto potrebbero quindi puntare a migliorare il detector, effettuando altri esperimenti per cercare di migliorare le metriche. Per fare questo, a disposizione di maggiori risorse di calcolo, si potrebbe tornare alla fase di campionamento del training set e utilizzarlo con un `sample_rate` minore oppure pari a 1, ovvero utilizzando l'intero set. Ciò potrebbe portare a elevati miglioramenti di performance sia per la detection che, di conseguenza, per il tracking.

Bibliografia

- [1] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. Bytetrack: Multi-object tracking by associating every detection box, 2022.
- [2] Chongwei Liu, Haojie Li, Zhihui Wang, and Rui Xu. Is a pure transformer effective for separated and online multi-object tracking?, 2025.
- [3] Longyin Wen, Dawei Du, Zhaowei Cai, Zhen Lei, Ming-Ching Chang, Honggang Qi, Jongwoo Lim, Ming-Hsuan Yang, and Siwei Lyu. Ua-detrac: A new benchmark and protocol for multi-object detection and tracking, 2020.
- [4] Arne Hoffhues Jonathon Luiten. Trackeval. <https://github.com/JonathonLuiten/TrackEval>, 2020.
- [5] Glenn Jocher, Jing Qiu, Mengyu Liu, Shuai Lyu, Fatih Cagatay Akyon, and Muhammet Esat Kalfaoglu. Ultralytics yolo26: Unified real-time end-to-end vision models, 2026.
- [6] Tsung-Yi Lin, Michael Maire, Serge Belongie, Lubomir Bourdev, Ross Girshick, James Hays, Pietro Perona, Deva Ramanan, C. Lawrence Zitnick, and Piotr Dollár. Microsoft coco: Common objects in context, 2015.